深度强化学习知识点复习（价值学习部分）

本文档参照王树森深度强化学习进行补充整理

2026 年 3 月 10 日

1 训练神经网络总是用 SGD 而非 GD 原因

- 梯度下降（GD）在处理非凸问题时容易陷入鞍点，难以收敛到局部最优；而随机梯度下降（SGD）和小批量 SGD 由于梯度的随机性，有助于跳出鞍点，更可能趋近局部最优，且泛化能力更强。
- GD 每一步需要计算整个数据集上的梯度，计算量是 SGD 的 n 倍（n 为样本数量），通常非常慢，除非使用大规模并行计算；SGD 每次仅用一个或一批样本更新，计算效率高，适合在线学习。

2 Robbins-Monro 算法

Robbins-Monro 算法是一种随机近似方法，用于在无法直接计算期望时在线更新参数。其更新形式为：

$$\theta_t = \theta_{t-1} + \alpha_t\big(f(x_t) - \theta_{t-1}\big) = (1-\alpha_t)\theta_{t-1} + \alpha_t f(x_t) \tag{1}$$

其中 α_t 是步长序列。为保证均方收敛，α_t 需满足：

$$\sum_{t=1}^{\infty} \alpha_t = \infty, \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty \tag{2}$$

该条件确保了算法能克服噪声并收敛到真实期望值，在 Q-learning 等强化学习算法中被用于更新动作价值函数。

3 对比动作价值函数、最优动作价值函数和状态价值函数的依赖因素

- **动作价值函数** $Q_\pi(s,a)$ 依赖于当前状态 s、当前动作 a 和策略 π，因为它衡量在策略 π 下从 (s,a) 出发的期望回报。
- **最优动作价值函数** $Q^*(s,a) = \max_\pi Q_\pi(s,a)$ 通过选择最优策略，只依赖于当前状态 s 和当前动作 a，与具体策略无关。
- **状态价值函数** $V_\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[Q_\pi(s,a)]$ 将动作视为随机变量求期望，因此只依赖于策略 π 和当前状态 s，不依赖于具体动作。

4 价值学习中 DQN 算法训练框架

DQN 使用时序差分（TD）算法学习最优动作价值函数，其理论基础是最优贝尔曼方程：

$$Q^*(s,a) = \mathbb{E}_{s'\sim P(\cdot|s,a)}\left[R + \gamma \max_{a'} Q^*(s',a')\right] \tag{3}$$

训练时通过深度神经网络 $Q(s,a;\theta)$ 近似 Q^*，并从经验回放池 $\mathcal{D}$ 中采样转移 (s,a,r,s')，最小化损失函数：

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta)\right)^2\right] \tag{4}$$

其中 θ^- 为目标网络参数（定期从 θ 复制），用于稳定训练。DQN 是异策略（off-policy）方法，允许使用任意行为策略收集数据并反复利用。

5 表格形式的 Q 学习训练框架

表格 Q 学习直接基于最优贝尔曼方程进行在线更新。对于每个转移 (s,a,r,s')，更新规则为：

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right] \tag{5}$$

其中 α 是学习率，γ 是折扣因子。该更新使用当前估计的 Q 值逼近最优贝尔曼方程中的期望，无需模型，属于异策略方法，因为更新中使用的 max 操作隐含了目标策略为贪婪策略，而与行为策略无关。

6 同策略 on-policy 与异策略 off-policy 方法比较

在强化学习中，用于与环境交互收集经验的策略称为**行为策略**，而最终要学习优化的策略称为**目标策略**。若二者相同，则为同策略（如 SARSA）；若不同，则为异策略（如 Q-learning）。异策略的最大优势是可以通过经验回放池存储历史数据并反复使用，提高样本效率；同策略则要求每次更新必须使用当前策略新采集的数据，样本利用率较低，但通常收敛更稳定，方差更小。

7 价值学习中表格形式的 SARSA 算法训练框架

SARSA 基于贝尔曼方程对当前策略 π 下的动作价值函数进行估计：

$$Q_\pi(s,a) = \mathbb{E}_{s'\sim P(\cdot|s,a),\, a'\sim\pi(\cdot|s')}\left[R + \gamma Q_\pi(s',a')\right] \tag{6}$$

对于每个五元组 (s,a,r,s',a')（其中 a' 是根据当前策略 π 在 s' 下选择的动作），SARSA 采用以下在线更新：

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma Q(s',a') - Q(s,a)\right] \tag{7}$$

SARSA 是同策略方法，因为更新中使用的 a' 必须由当前策略生成，若使用经验回放，则存储的 a' 可能来自旧策略，直接使用会导致更新方向偏离当前策略的贝尔曼方程，因此不能简单地应用经验回放。若需重用数据，必须结合重要性采样等技术，但基本 SARSA 不支持。

8 价值学习中 Q 学习和 SARSA 的对比

- **更新公式**：Q 学习使用 $\max_{a'} Q(s', a')$（公式 (5)），直接逼近最优价值函数；SARSA 使用当前策略下的动作 a' 对应的 $Q(s', a')$（公式 (7)），逼近当前策略的价值函数。
- **策略依赖性**：Q 学习与行为策略无关（异策略），可使用任意策略收集的数据；SARSA 严格依赖于当前策略（同策略），更新必须使用当前策略选择的动作。
- **经验回放**：Q 学习可配合经验回放，提高样本效率；SARSA 不能直接使用经验回放，因为旧经验中的 a' 不再反映当前策略。
- **收敛性**：在表格情形下，两者都能收敛到最优值函数（Q 学习）或给定策略的值函数（SARSA），但 SARSA 的收敛通常更平稳，而 Q 学习可能因最大化偏差导致过估计。
- **方差与偏差**：SARSA 使用实际采样的 a'，方差较小但可能陷入次优；Q 学习使用最大化操作，偏差较小但方差较大，且容易引入正向偏差。

9 价值学习中神经网络形式的 SARSA 算法

当状态空间为无限集合时，可采用神经网络 $Q(s, a; \theta)$ 来近似动作价值函数 $Q_\pi(s, a)$，其中 θ 为网络参数。神经网络 SARSA 仍为同策略方法，需使用当前策略 π 采集五元组 (s, a, r, s', a')，其中 $a' \sim \pi(\cdot|s')$。其训练目标是最小化 TD 误差的平方：

$$L(\theta) = \mathbb{E}_{(s,a,r,s',a')\sim\mathcal{D}} \left[\big(r + \gamma Q(s', a'; \theta) - Q(s, a; \theta)\big)^2 \right] \tag{8}$$

参数更新采用梯度下降：

$$\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta) \tag{9}$$

注意，由于 SARSA 是同策略，经验回放池 $\mathcal{D}$ 中存储的转移必须严格由当前策略生成，否则更新公式 (8) 不再对应当前策略的贝尔曼方程。在实际应用中，神经网络 SARSA 常与策略网络结合构成 Actor-Critic 方法：策略网络 $\pi(a|s; \phi)$ 作为 Actor 控制智能体，价值网络 $Q(s, a; \theta)$ 作为 Critic 评价当前策略并指导 Actor 更新。此时 Critic 仍采用 SARSA 方式更新，即基于实际采样的动作 a' 计算 TD 目标。

10 多步 TD 算法的原理推导

多步 TD（n 步 TD）算法利用未来 n 步的累积奖励来更新价值函数，以平衡偏差与方差。其基于多步回报的贝尔曼方程：

$$Q_\pi(s_t, a_t) = \mathbb{E}_{s_{t+1}, a_{t+1}, \ldots, s_{t+n}} \left[\sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q_\pi(s_{t+n}, a_{t+n}) \right] \tag{10}$$

定义 n 步回报：

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q(s_{t+n}, a_{t+n}; \theta) \tag{11}$$

则参数更新通过最小化 TD 误差实现：

$$\theta \leftarrow \theta + \alpha \left(G_t^{(n)} - Q(s_t, a_t; \theta) \right) \nabla_\theta Q(s_t, a_t; \theta) \tag{12}$$

训练流程：在每个时刻 t，智能体与环境交互并存储转移，当收集到 n 步数据后，利用公式 (11) 计算目标值，再按 (12) 更新网络。n 步 TD 介于蒙特卡洛（$n \to \infty$）和一步 TD（$n = 1$）之间，可通过调整 n 控制偏差-方差权衡。

11 训练价值网络采用蒙特卡洛方法的优劣性

蒙特卡洛方法使用完整回报 $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ 作为目标，其优势是**无偏估计**：由于回报直接来自真实轨迹，期望等于真实价值函数。但劣势是**方差大**，因为单条轨迹的随机性（如策略随机、环境转移随机）导致回报波动剧烈，且需等待回合结束才能更新，学习效率低。

12 训练价值网络采用自举方法的优劣性

自举方法（如一步 TD）使用当前估计的后续价值构建目标，即 $r + \gamma Q(s', a'; \theta)$。其优势是**方差小**，因为目标只依赖单步随机变量；劣势是**有偏估计**，因为目标中包含尚未收敛的当前价值估计，导致更新方向存在偏差。自举使得算法可在线更新，但可能引入累积误差。

13 蒙特卡洛方法与自举的对比

- **偏差**：蒙特卡洛无偏；自举有偏（依赖于当前估计的准确性）。
- **方差**：蒙特卡洛高方差（需积分多步随机性）；自举低方差（仅涉及单步随机性）。
- **更新时机**：蒙特卡洛需回合结束；自举可每一步更新。
- **收敛性**：蒙特卡洛在表格情形下保证收敛，但收敛慢；自举可能加速收敛，但可能不收敛（尤其在函数近似时，如 Q-learning 可能发散）。
- **应用场景**：蒙特卡洛适用于回合制任务；自举适用于连续任务，常与函数近似结合。

14 经验回放的优点

1. **打破序列相关性**：强化学习数据在时间上高度相关，直接使用连续样本更新违背了多数优化算法对独立同分布的假设。经验回放随机抽取历史转移，使得样本近似独立同分布，稳定训练。
2. **提高样本效率**：每个转移可被多次使用，从有限交互中学习更多信息，减少与环境交互的总次数，尤其适用于数据获取成本高的场景。

15 样本数量与更新次数的区别

样本数量指智能体与环境交互产生的转移总数（即采集的数据量），**更新次数**指利用这些样本对参数进行梯度下降的次数。在经验回放中，一个样本可被多次更新。样本数量更为关键，因为它决定了数据覆盖的状态-动作空间的广度；更新次数虽可增加，但过度重复使用有限样本可能导致过拟合和泛化能力下降。因此，收集更多样本来覆盖更广分布通常比单纯增加更新次数更有效。

16 经验回放的局限性

1. **策略过时问题**：回放池中存储的转移由旧行为策略生成，与当前待学习的目标策略可能存在分布偏移。这可能导致更新方向偏离最优解，尤其在策略敏感算法（如同策略方法）中。

2. **仅适用于异策略**：经验回放本质上要求目标策略与行为策略可以不同，因为存储的数据来自过去策略。同策略算法（如 SARSA）若使用回放池，则存储的 a' 不再反映当前策略，直接更新会违背其理论基础。因此经验回放通常与异策略算法（如 DQN、Q 学习）结合使用。

17 优先经验回放原理及参数更新方式

优先经验回放（PER）通过赋予经验样本不同的采样概率，使智能体更频繁地学习那些“意外性”高的样本，通常用 TD 误差衡量。其核心思想是：TD 误差绝对值越大，表明当前 Q 网络对该样本的预测越不准确，该样本应具有更高的采样优先级。

定义样本 i 的优先级 $p_i > 0$，常用两种形式：

- 比例优先级：$p_i = |\delta_i| + \epsilon$，其中 δ_i 为 TD 误差，ϵ 是小常数防止概率为零。
- 基于排序的优先级：$p_i = \frac{1}{\text{rank}(i)}$，其中 $\text{rank}(i)$ 是按 $|\delta_i|$ 排序的序号。

采样概率由优先级归一化得到：

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \tag{13}$$

其中 $\alpha \in [0, 1]$ 控制优先级的使用程度，$\alpha = 0$ 退化为均匀采样。

由于采样分布偏离原始经验分布，为无偏估计梯度，需乘以重要性采样权重：

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^\beta$$

其中 N 为回放缓冲区大小，β 从某个较小值（如 0.4）逐渐增至 1，以补偿偏差。实际更新时通常将权重归一化：$w_i / \max_j w_j$ 以提高稳定性。

在 DQN 中，使用优先经验回放时，损失函数为：

$$L(\theta) = \frac{1}{B} \sum_{i \in \text{batch}} w_i \left(r_i + \gamma \max_{a'} Q(s_i', a'; \theta^-) - Q(s_i, a_i; \theta)\right)^2 \tag{14}$$

其中 B 为 batch 大小，θ^- 为目标网络参数。每次更新后，需要重新计算被采样样本的 TD 误差，并更新其优先级 p_i。

18 优先经验回放通过调整学习率修正

优先经验回放引入的非均匀采样会改变损失函数的期望，导致有偏估计。重要性采样权重 w_i 的作用就是修正这一偏差：通过将每个样本的梯度乘以 w_i，使得更新方向近似于原始均匀采样下的期望梯度。

从参数更新角度看，这等价于对每个样本使用了自适应的学习率。设原始学习率为 η，则实际更新步长为 $\eta w_i \nabla L_i$。高优先级样本的 $P(i)$ 大，对应的 w_i 小，从而降低了它的更新步长；低优先级样本的 w_i 大，步长相对提高。这样，虽然采样更频繁地偏向高优先级样本，但权重调整防止了这些样本过度主导更新，使整体更新更接近均匀采样的效果。

19 大抽样概率、小学习率结合不会导致效果抵消原因分析

高优先级样本被频繁采样（大抽样概率），但其权重 w_i 较小（小学习率），二者相乘会不会导致最终更新量不变，从而使优先采样的效果被抵消?

关键在于，优先采样改变的是样本被选中的频率，而权重调整的是每次更新的幅度。最终每个样本对参数更新的总贡献期望为：

$$\mathbb{E}_{\text{PER}}[w_i\Delta_i] = \sum_i P(i)\cdot w_i\Delta_i = \sum_i P(i)\cdot\frac{1}{NP(i)}\Delta_i = \frac{1}{N}\sum_i \Delta_i = \mathbb{E}_{\text{uniform}}[\Delta_i] \tag{15}$$

其中 Δ_i 为未加权的梯度。这说明，在期望意义上，PER 与均匀采样等价，即无偏性成立。然而，方差却降低了：高 TD 误差的样本贡献被分摊到更多更新步中，而低 TD 误差的样本贡献被集中，从而加速了学习。因此，大抽样概率和小学习率的结合并未抵消效果，反而在保持期望不变的前提下，通过减小方差提升了效率。

20 Q 学习高估真实价值的原因

Q 学习及其深度变种（如 DQN）常出现对动作价值的过高估计，主要原因有以下两点。

1. **自举导致偏差传播**自举（bootstrapping）指用当前的估计值来更新自身。在 Q 学习中，更新目标为：

$$Y_t = R_{t+1} + \gamma\max_a Q(S_{t+1}, a; \theta^-) \tag{16}$$

其中目标网络 θ^- 本身也是估计值，包含误差。如果某次更新使 Q 被高估，那么后续使用该高估值作为目标时，误差会进一步传播放大。数学上，假设真实值 Q^*，估计值 $\hat{Q}$，初始误差 ϵ_0，则经过一步更新：

$$\hat{Q}_{\text{new}}(s,a) = \hat{Q}(s,a) + \alpha\left(r + \gamma\max_{a'}\hat{Q}(s',a') - \hat{Q}(s,a)\right) \tag{17}$$

若 $\max_{a'}\hat{Q}(s',a') > \max_{a'}Q^*(s',a')$，则目标偏高，导致 $\hat{Q}_{\text{new}}$ 进一步高估。这种偏差通过贝尔曼方程迭代传播，最终可能导致严重的过高估计。

2. **最大化导致 TD 目标高估真实价值**即使初始估计无偏，最大化操作也会引入正向偏差。设对于某个状态 s'，真实值为 $Q^*(s',a)$，而估计值 $Q(s',a) = Q^*(s',a) + \varepsilon_a$，其中 ε_a 是均值为 0 的独立噪声。则：

$$\mathbb{E}\left[\max_a Q(s',a)\right] \geq \max_a \mathbb{E}[Q(s',a)] = \max_a Q^*(s',a) \tag{18}$$

严格不等式成立通常因为最大值函数的期望大于期望的最大值。更具体地，假设所有动作的真实值相等（设为 V^*），噪声 ε_a 独立同分布，则：

$$\mathbb{E}\left[\max_a(V^* + \varepsilon_a)\right] = V^* + \mathbb{E}[\max_a \varepsilon_a] \geq V^* \tag{19}$$

且 $\mathbb{E}[\max_a \varepsilon_a] > 0$（只要分布非退化）。因此，即使噪声均值为零，取最大值后期望也会高估。

21 价值函数均匀高估与非均匀高估的影响

若所有动作价值被均匀高估（即每个 $Q(s,a)$ 都增加相同常数），则贪心策略不变，不会影响决策。但实际中高估是非均匀的：不同动作的噪声不同，且随着学习过程，某些动作可能被高估更多。这会导致：

- 次优动作因高估而被选中，使策略变差。
- 高估在自举中传播，使状态价值整体膨胀，破坏学习的稳定性。
- 最终可能使算法无法收敛到最优策略。

22 使用目标网络切断自筹误差传播

自举（bootstrapping）是指用当前价值网络的估计值去更新自身，这会导致误差的传播和累积，使训练不稳定甚至发散。目标网络通过提供固定的 TD 目标来切断这一环路。

设在线网络参数为 θ，目标网络参数为 θ^-，两者结构相同但 θ^- 定期从 θ 复制（硬更新）或通过软更新逐步逼近。DQN 的 TD 目标变为：

$$Y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) \tag{20}$$

由于 θ^- 在 C 步内保持不变，计算目标时使用的 Q 值不依赖于当前 θ，从而避免了用刚刚更新的网络立即评估自身。这相当于将目标固定一段时间，使训练更稳定。

但需注意：目标网络只是延迟了自举，并未完全消除偏差，因为 θ^- 本身仍是旧参数的估计值，依然包含误差，只是更新频率降低。从理论上讲，目标网络使贝尔曼更新变成了一个半梯度（semi-gradient）方法，牺牲了一定无偏性换取了稳定性。

23 双 Q 学习算法原理

标准 Q 学习使用 max 操作同时进行动作选择和价值评估，容易导致过高估计。Double Q-learning 将选择和评估解耦，用两个独立的 Q 函数来减少正偏。

在 Double DQN 中，动作选择由在线网络 θ 完成，而价值评估由目标网络 θ^- 完成，目标为：

$$Y_t = r_t + \gamma Q\big(s_{t+1}, \arg\max_{a'} Q(s_{t+1}, a'; \theta); \theta^-\big) \tag{21}$$

即先用 θ 选出最优动作 $a^* = \arg\max_{a'} Q(s_{t+1}, a'; \theta)$，再用 θ^- 评估该动作的价值。这样即使 θ 高估了某个动作，θ^- 的评估相对独立，可缓解高估。

参数更新仍通过最小化 TD 误差平方：

$$L(\theta) = \mathbb{E}\left[\big(Y_t - Q(s_t, a_t; \theta)\big)^2\right] \tag{22}$$

理论上，Double DQN 能显著降低过高估计，使价值估计更准确，从而改善策略。

24 最优优势函数

优势函数 $A^\pi(s,a)$ 定义为动作价值与状态价值之差，表示采取动作 a 相对于平均水平的优势：

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s) \tag{23}$$

对于最优策略，记最优优势函数为 $A^*(s,a) = Q^*(s,a) - V^*(s)$。根据最优价值函数的定义，$V^*(s) = \max_a Q^*(s,a)$，代入可得：

$$\max_a A^*(s,a) = \max_a Q^*(s,a) - V^*(s) = 0 \tag{24}$$

即最优优势函数在最优动作上的值为 0。这一性质为后续对决网络的设计提供了理论基础：可以将 Q 值分解为状态价值加上优势函数，并通过对优势函数施加约束来保证分解的唯一性。

25 对决网络结构与原理

对决网络（Dueling Network）是对 DQN 架构的改进，它将 Q 值显式分解为状态价值 $V(s)$ 和优势函数 $A(s,a)$ 两部分：

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + A(s,a;\theta,\alpha) \tag{25}$$

其中 θ 是卷积层等共享参数，β 和 α 分别为价值流和优势流的参数。这种结构使网络能分别学习状态价值和动作优势，在某些情况下（如某些动作对价值影响不大时）收敛更快、估计更准。

其原理是：在训练过程中，价值流会学习状态的基准价值，而优势流则学习每个动作相对于基准的偏移。通过共享底层特征，两部分相互促进，提高了表示效率。实验表明，对决网络在同样参数规模下优于传统 DQN。

26 解决不唯一性，max D(s,a,w) 不能省略

上述分解存在**不唯一性**：若将 V 加上任意常数 c，同时将 A 减去 c，则 Q 保持不变。这导致网络无法确定 V 和 A 的取值，可能产生无意义的波动，影响收敛。

根据 $\max_a A^*(s,a) = 0$ 的性质，可以强制优势函数在最优动作处为零，从而消除不唯一性。一种常用做法是：

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \Big(A(s,a;\theta,\alpha) - \max_{a'} A(s,a';\theta,\alpha)\Big) \tag{26}$$

这样 $\max_a Q(s,a) = V(s)$，保证了 V 确实是状态价值，A 唯一地表示优势。此处的 max 项不能省略，否则无法施加约束，分解仍然不唯一。

27 实际实现对决网络时为什么用 mean 替代 max

尽管理论公式使用 max 操作，但实践中通常用均值替代最大值：

$$Q(s,a) = V(s) + \left(A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a')\right) \tag{27}$$

原因如下：

- **梯度稳定性**：max 操作会使梯度只更新被选中的动作对应的优势，其他动作的优势梯度为 0，导致更新不稳定；而均值使所有动作的优势都参与梯度更新，更加平滑。
- **收敛速度**：均值方式下，优势函数倾向于更快地收敛到相对值，且实验表明整体收敛更快。
- **等价性**：虽然此时 $V(s)$ 不再严格等于 $\max_a Q(s,a)$，但只是整体偏移了一个常数，不影响动作选择的相对顺序，因此对最终策略无影响。

28 噪声网络原理

噪声网络（NoisyNet）是一种替代 ϵ-greedy 的探索策略，通过在网络参数上添加参数化噪声来驱动探索，使探索成为网络的一部分并随训练自适应调整。

对于线性层 $y = wx + b$，噪声网络将其参数化为：

$$w = \mu^w + \sigma^w \odot \varepsilon^w, \quad b = \mu^b + \sigma^b \odot \varepsilon^b \tag{28}$$

其中 μ 和 σ 是可学习参数，ε 是零均值噪声（如标准正态分布），$\odot$ 为逐元素乘。前向传播时，采样 ε 得到实际参数，从而输出的 Q 值带有随机性，驱动探索。噪声的方差由 σ 控制，网络可通过调整 σ 自适应地控制探索程度：初期 σ 较大，探索强；后期 σ 可能减小，探索减弱。

梯度反向传播时，只更新 μ 和 σ，ε 不参与更新。噪声的引入使探索与网络融为一体，避免了手工调参 ϵ，且在理论上表现更好。

29 噪声 DQN 实现框架

噪声 DQN 将原始 DQN 中的全连接层替换为噪声层，实现步骤如下：

1. 对每个要替换的线性层，定义可学习参数 $\mu \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 和 $\sigma \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$（偏置同理）。
2. 每次前向传播时，采样噪声 $\varepsilon^w \sim \mathcal{N}(0,1)$ 和 $\varepsilon^b \sim \mathcal{N}(0,1)$，计算实际权重和偏置：

$$w = \mu^w + \sigma^w \odot \varepsilon^w, \quad b = \mu^b + \sigma^b \odot \varepsilon^b$$

3. 使用噪声参数进行前向计算得到 Q 值。
4. 训练时，损失函数与 DQN 相同（如公式 (4)），梯度反向传播更新 μ 和 σ。
5. 探索由参数的随机性自动产生，无需 ϵ-greedy。

实践中，为减少计算量，常采用**因子化噪声**：每个单元独立采样一个噪声，再通过线性变换构造参数噪声，避免为每个参数独立采样。